How Quant Signals Works — From Scratch

A guided tour of the whole system for someone who has never built a trading model. It assumes you can read Python but explains every finance and machine-learning concept as it comes up. Read top to bottom; each part builds on the last. Code excerpts are the real code, lightly trimmed.

---

0. The one-paragraph version

Every week the system downloads stock prices, turns them into a handful of numerical **features** per stock (momentum, volatility, distance from the 52-week high, …), feeds those to a model that **ranks** the ~500 S&P 500 stocks from most to least attractive over the next ~2 weeks, "buys" the top ~10% on paper, and writes those positions to a file **before** anyone knows how they turn out. Later it checks how they actually did. Everything else — the backtest, the dashboard, the research rounds — exists to answer one question honestly: does this ranking contain any real predictive signal, or are we fooling ourselves?

The whole design is built around the fact that **it is extremely easy to fool yourself** in this domain, and the forward paper record is the one referee that can't be fooled.

---

1. The core idea: ranking, not fortune-telling

A naive trading model tries to predict "will the market go up tomorrow?" That's nearly impossible and this system deliberately does **not** try.

Instead it predicts **relative** performance: of these 500 stocks, which ones will beat the others over the next ~2 weeks? This is a **cross- sectional** strategy — "cross-section" meaning "all the stocks at one point in time." You don't need to know if the market goes up or down; you need the stocks you rank highly to do better than the ones you rank low. You buy the top-ranked names in equal amounts and you're betting on the spread between good and bad picks.

Why this is a smarter problem to attack: - Market direction is dominated by macro noise nobody can forecast. - Relative performance has real, documented drivers (called **anomalies** or **factors**): stocks that have been rising tend to keep rising for a while (momentum), very volatile stocks tend to underperform, etc. - If every stock drops 5% in a crash, a ranking strategy can still "win" by having its longs drop less than its shorts — the market move cancels out.

The catch: these edges are tiny. A good stock-ranking model has a correlation with future returns of maybe 0.02–0.05 (you'll see this number, called **IC**, on the dashboard). That is not a typo. The entire game is harvesting a very small, very noisy edge without letting costs and self-deception eat it.

---

2. The data layer — `quant_signals/data/`

2.1 Who's in the game: `universe.py`

The **universe** is the list of stocks you're allowed to trade. Here it's the current S&P 500, scraped from Wikipedia:

```
def get_sp500_tickers() -> list[str]:
    return tickers_from_table(_fetch_tables(WIKI_URL)[0])
```

`tickers_from_table` does one fiddly-but-important thing — it normalizes ticker symbols to the format Yahoo Finance expects (`BRK.B` → `BRK-B`) and drops duplicates.

The hidden trap here is survivorship bias. The current S&P 500 is a list of winners — companies that grew enough to be in the index today. Companies that went bankrupt or got kicked out aren't in the list, so a backtest over the current universe never sees the losers that existed in 2018. This makes every historical result look better than reality. The system can't fully fix this (it would need expensive point-in-time constituent data), so instead it's **honest about it**: the dashboard and status page label results "survivorship-biased," and there's a "veterans-only" robustness check (stocks with full history) to size the distortion.

2.2 The prices: `prices.py`

Prices come from Yahoo Finance via the `yfinance` library, into two wide tables saved as Parquet files (a fast binary format): - `closes.parquet` — rows are dates, columns are tickers, cells are the **adjusted** closing price (adjusted = dividends and stock splits already baked in, so a 2-for-1 split doesn't look like a 50% crash). - `volumes.parquet` — same shape, share volumes.

The clever part is `update_price_cache` + `_download_incremental`. On the first run it downloads 10 years of history. On later runs it only re-fetches the last ~7 days for stocks it already has (fast), and downloads full history only for stocks newly added to the index. The merge uses `combine_first`:

```
def merge_frames(old, new):
    """New values win where new has data; a missing cell in `new`
    never erases real values in `old`."""
    return new.combine_first(old).sort_index()
```

That comment is scar tissue — an earlier row-level version once wiped history with NaNs. **Lesson embedded in the code: your data cache is precious; never let a bad download destroy good history.**

---

3. Features — turning prices into signal

A model can't eat raw prices. It eats **features**: numbers that summarize something potentially predictive about each stock on each day. All the feature math lives in `features/indicators.py`, and each function operates on the whole price table at once (every stock, every day, vectorized).

3.1 The indicators, one at a time

```
def momentum(closes, window):
    base = closes.shift(window)
    return (closes / base - 1).mask(base <= 0)
```

Momentum = the return over the last `window` trading days. `closes / closes.shift(21) - 1` is "today's price divided by the price 21 days ago, minus 1" = the 21-day return. The `.mask(base <= 0)` blanks out any bad data where the old price was zero or negative. The system computes momentum over 21, 63, 126, and 252 days (roughly 1, 3, 6, 12 months). Why multiple windows? Different horizons capture different behavior — 12-month momentum is the classic anomaly; 5-day "momentum" is actually short-term **reversal** (recent winners tend to dip back), which is why the baseline weights it negatively.

```
def rsi(closes, window=14):
    delta = closes.diff()
    gain = delta.clip(lower=0).rolling(window).mean()
    loss = (-delta.clip(upper=0)).rolling(window).mean()
    return 100 - 100 / (1 + gain / loss)
```

RSI (Relative Strength Index) measures how one-sided recent moves have been, on a 0–100 scale. It splits each day's price change into "gains" and "losses," averages each over the window, and forms a ratio. ~70+ means "mostly up lately," ~30- means "mostly down." The edge case — a window with zero losses would divide by zero — is handled by forcing RSI to 100 there.

```
def realized_vol(closes, window):
    return closes.pct_change().rolling(window).std() * np.sqrt(TRADING_DAYS)
```

Realized volatility = how jumpy the stock has been. Take daily percent changes, compute their standard deviation over the window, and annualize by multiplying by √252 (the standard trick to turn a daily wiggle into a yearly one — variance scales

with time, so standard deviation scales with its square root). Low-volatility stocks tend to outperform, so the baseline weights this negatively.

`sma_distance` (how far above/below a moving average), `high_proximity` (how far below the 52-week high — stocks near their highs tend to keep climbing), and `volume_zscore` (is today's volume unusually high vs. its own recent average) round out the set. Twelve features total, listed in `panel.FEATURES`.

3.2 Assembling the panel and the crucial z-score: `panel.py`

`build_feature_panel` computes all twelve indicators and stacks them into one long table indexed by `(date, ticker)`, one column per feature, dropping any row with a missing value (a stock without enough history yet).

Then comes the single most conceptually important line in the feature code:

```
def zscore_by_date(panel):
    g = panel.groupby(level="date")
    z = (panel - g.transform("mean")) / g.transform("std")
    return z.clip(-5, 5)
```

Z-scoring by date means: on each day, for each feature, subtract the average across all stocks and divide by the standard deviation across all stocks. After this, a feature value of `+1.5` means "1.5 standard deviations above the average stock today," not any absolute quantity.

Why this matters enormously: it makes the model learn **cross-sectional rank**, not market level. In a bull market every stock's momentum is positive; z-scoring removes that shared component and leaves only "which stocks have more momentum than their peers right now." The clip to ±5 stops a single crazy outlier from dominating. This is the mechanical embodiment of Part 1 — the whole system thinks in relative terms, and this is where "relative" is enforced.

3.3 The label — what the model is trained to predict

Features are the input. The **label** is the answer key: what actually happened next, which the model learns to predict. This is the most dangerous code in any trading system, because a subtle mistake here leaks the future into the past and produces a beautiful backtest that makes no money live. Read it slowly:

```
def forward_demeaned_returns(closes, horizon=10):
    entry = closes.shift(-1)              # price the day AFTER the signal
    exit_ = closes.shift(-(1 + horizon))  # price 10 trading days after entry
    fwd = exit_ / entry - 1               # the return we'd have captured
    demeaned = fwd.sub(fwd.mean(axis=1), axis=0)   # subtract the day's average
    ...
```

Two deliberate choices:

1. `shift(-1)` **— enter the day after the signal.** The signal for date t is computed from prices up to and including t's close. You cannot trade at that same close — it already happened. So the position is assumed to be entered at t+1's close and held to t+1+10. No price at or before t appears in the return the model is graded on. This one `-1` is the wall between "information you had" and "returns you earned." Get it wrong (use `shift(0)`) and you've told the model the future.
2. `demeaned` **— subtract each day's cross-sectional average return.** If the whole market rose 3% over the window, every stock's raw return is inflated by ~3%. Subtracting the day's mean strips out the market move and leaves only outperformance. The model is thus trained to predict relative winners, matching exactly what z-scored features encode and what the strategy actually bets on. **Input and label agree: both are market-neutral.**

`horizon=10` (`config.LABEL_HORIZON`) means the model predicts ~2-week-ahead relative returns, which is why positions turn over roughly every 2 weeks.

---

4. The models — three scorers

The system runs three scorers side by side so it always has a control group. A "score" is just a number per `(date, ticker)`; higher = more attractive.

4.1 Baseline — the honest hurdle: `baseline.py`

```
BASELINE_WEIGHTS = {"mom_63": 0.15, "mom_126": 0.20, "mom_252": 0.15,
                    "rev_5": -0.10, "vol_63": -0.15, "high_252": 0.15,
                    "sma_200": 0.10}

def baseline_scores(zpanel):
    return sum(w * zpanel[f] for f, w in BASELINE_WEIGHTS.items())
```

That's the entire baseline: a **fixed weighted sum** of z-scored features, with signs chosen from decades-old published finance research (momentum and 52-week-high positive; short-term reversal and volatility negative). No learning, no fitting. Its job is to be the bar the fancy model must clear. If a machine-learning model with millions of parameters can't beat seven hand-set weights, the machine learning is worthless. (Spoiler from the research rounds: the ML model beats it, but only by a little.)

4.2 The ML scorer — where the real subtlety lives: `ml.py`

The ML model is a **gradient-boosted regression tree** (`HistGradientBoostingRegressor` from scikit-learn). At a high level: it's a machine that builds hundreds of small decision trees, each one correcting the errors of the ones before it, to learn a flexible non-linear function from the twelve features to the label. You

don't need the internals — treat it as "a smart curve-fitter that can capture interactions the linear baseline can't."

The hard part isn't the model, it's **how it's trained without cheating.** You cannot train on all of history and test on the same history — the model would just memorize. And you must never train on data from after the point you're predicting. The solution is **walk-forward training with purging**:

```
def walk_forward_splits(dates, train_window_days=756, purge_days=10,
                        min_train_days=252):
    for month in dates.to_period("M").unique():
        score_dates = dates[dates.to_period("M") == month]
        first = dates.get_loc(score_dates[0])
        end = first - purge_days - 1          # exclusive end of train slice
        start = max(0, end - train_window_days)
        if end - start < min_train_days:
            continue
        yield dates[start:end], score_dates
```

Walk through it: - For **each calendar month** of dates you want to score… - `first` is the index of that month's first day. - The training data ends at `end = first - purge_days - 1`, i.e. **11 days before** the month starts. Training uses `train_window_days` = 756 trading days (~3 years) before that. - So the model that scores July was trained only on data ending in mid-June and stretching back 3 years. It genuinely never saw July.

Why the `purge_days` gap? This is the non-obvious masterstroke. Recall the label spans 10 future days. The last training day's label already peeks 10 days into the future. Without a gap, that future would overlap the scoring period — a leak. Purging 10+1 days guarantees the newest training label ends before the scored period begins. It's the temporal equivalent of washing your hands between the training kitchen and the testing kitchen.

The model is **retrained every month** (`_make_model` builds a fresh one), walking forward through time, always trained on the recent past, always scoring the untouched near-future. This is how you get an honest, out-of-sample score for every historical date — the backbone of a trustworthy backtest.

4.3 Blend — combining the two: `blend.py`

```
def blend_scores(ml, baseline):
    def pct(s): return s.groupby(level="date").rank(pct=True)
    joined = pd.concat([pct(ml), pct(baseline)], axis=1).dropna()
    return joined.mean(axis=1)
```

The blend converts each scorer's output into **percentile ranks within each day** (so the #1 stock is 1.0, the median is 0.5) and averages them. Ranking first makes the two incomparable score scales comparable. The blend is a diversification bet — two mediocre-but-different signals averaged can be steadier than either alone. (In

the robustness checks, the blend often wins the "most robust" title even when ML wins raw Sharpe.)

---

5. The backtest engine — `backtest/engine.py`

This simulates actually trading the scores through history, with the two mechanisms that separate a toy backtest from an honest one: **buffer bands** (to control trading costs) and a **regime filter** (to control crash risk). This is the function to understand line by line.

5.1 Buffer bands — the turnover killer

```
def _target_names(day_scores, held, top_fraction, exit_fraction):
    ranks = day_scores.rank(ascending=False, method="first")
    n_target = max(1, int(round(len(day_scores) * top_fraction)))
    exit_cutoff = max(n_target, int(round(len(day_scores) * exit_fraction)))
    keep = [t for t in held if t in ranks.index and ranks[t] <= exit_cutoff]
    n_fill = n_target - len(keep)
    if n_fill > 0:
        fill = ranks.drop(index=keep).nsmallest(n_fill).index.tolist()
    else:
        fill = []
    return pd.Index(keep + fill)
```

The naive strategy: "hold the top 10% every week." The problem: a stock ranked #50 one week and #52 the next flips in and out constantly, and every trade costs money. Over a year that **churn** — called **turnover** — quietly bleeds the strategy to death (this was the lesson of research round 2, which you'll see referenced as "trading costs eat alpha").

Buffer bands fix it with a **two-threshold** rule: - **Enter** a stock only if it's in the top `top_fraction` (10%). - But **keep** a stock you already hold as long as it stays in the top `exit_fraction` (20%) — a wider band.

So a stock has to fall out of the top 20% to be sold, not just out of the top 10%. Names near the boundary stop flip-flopping. The code: keep held names still inside the exit band (`ranks[t] <= exit_cutoff`), then fill the remaining slots up to the 10% target from the best unheld names. Round 2 showed this roughly **halved turnover** and lifted the Sharpe ratio from 0.85 to 1.05 — a bigger gain than any model improvement, from pure structure. The same `_target_names` function is imported by the paper ledger so live and backtest selection are provably identical.

5.2 The regime filter — crash insurance

```
def regime_exposure(benchmark_closes, window=200, scale=0.5):
    sma = benchmark_closes.rolling(window).mean()
```

```
    return pd.Series(1.0, index=benchmark_closes.index).mask(
        benchmark_closes < sma, scale)
```

When the market (SPY) closes **below its 200-day moving average** — the classic "we're in a downtrend" signal — this returns `0.5` instead of `1.0`, telling the backtest to hold only **half** the usual position size. Above the average, full exposure. It's a crude but effective bear-market seatbelt: it can't predict crashes, but it mechanically reduces exposure during the sustained downtrends where crashes happen. On the dashboard's Predictions tab, the amber shading marks the periods this filter went defensive. This is the model's only market-level opinion; everything else is relative.

5.3 Running the simulation

`run_backtest` steps through every trading day. The heart of the loop:

```
for date in sim_dates:
    ret = float((active * rets.loc[date].reindex(active.index)).sum())
    cost = 0.0
    if pending is not None:              # a rebalance from yesterday executes now
        traded = float(pending.sub(active, fill_value=0).abs().sum())
        turnover[date] = traded
        cost = traded * cost_bps / 1e4
        active = pending
        pending = None
    if date in rebal_dates:              # Friday: compute new target book
        day_scores = scores.xs(date, level="date").dropna()
        names = _target_names(day_scores, held, top_fraction, exit_fraction)
        held = names
        scale = float(exposure.loc[date]) if exposure is not None else 1.0
        pending = pd.Series(scale / len(names), index=names)
    daily.append(ret - cost)
```

The timing is the subtle part and it deliberately matches the label definition from §3.3: - `active` holds today's actual weights; `ret` is today's portfolio return (each holding's weight × its return today). - When a rebalance was decided yesterday (`pending` is set), it **executes today** at today's close. The **turnover** is the sum of absolute weight changes, and the **cost** is that turnover × `cost_bps`/10000. `cost_bps = 10` means 10 **basis points** = 0.10% charged on every unit of weight traded (a realistic estimate of commissions + spread + slippage). - On a **rebalance date** (Fridays, `REBALANCE_FREQ = "W-FRI"`), it computes the new target book via buffer bands, scales it by the regime filter, and stores it as `pending` — to execute next day. That one-day delay is the same `shift(-1)` wall as the label: you decide on Friday's close, you trade at Monday's close. - `daily.append(ret - cost)` records the net daily return.

The function returns the daily return series and the per-rebalance turnover. That daily series is what everything downstream measures.

---

6. Measuring performance — `backtest/metrics.py`

Three numbers summarize a return series. All assume 252 trading days/year.

```
def cagr(returns):
    total = float((1 + returns).prod())
    years = len(returns) / TRADING_DAYS
    return total ** (1 / years) - 1
```

CAGR (Compound Annual Growth Rate) — if you compounded these daily returns, what smooth yearly rate would produce the same final result? It's the "what % per year did this make" number. `(1+returns).prod()` compounds everything; then you take the appropriate root to annualize.

```
def sharpe(returns):
    return float(returns.mean() / returns.std() * np.sqrt(TRADING_DAYS))
```

Sharpe ratio — the single most important number in the whole system. It's average return divided by the volatility of return, annualized. It answers "how much return did you get per unit of risk?" A Sharpe of 1.0 is genuinely good for a stock strategy; SPY itself sits around 0.8. Two strategies making the same return aren't equal — the one with the smoother ride (lower standard deviation, higher Sharpe) is better, because you can hold it without panicking and can safely lever it. This is why the system optimizes and reports Sharpe, not raw return.

```
def max_drawdown(returns):
    curve = (1 + returns).cumprod()
    return float((curve / curve.cummax() - 1).min())
```

Max drawdown — the worst peak-to-trough fall along the equity curve. `cummax()` tracks the highest point reached so far; `curve / that - 1` is how far below the prior peak you are at each moment; `.min()` is the deepest that ever got. A max drawdown of −38% means "at the worst moment you were down 38% from your high." It's the number that tells you whether you could actually stomach holding the strategy.

---

7. The paper record — `paper.py`

This is the philosophical core. A backtest is a story you tell about the past, and stories can be massaged. The paper record is different: it writes down **real positions before the outcome exists**, to an append-only file committed to git. You can't edit the past in git without it being obvious. It's the one experiment nobody — not even you — can fudge.

7.1 Recording (before outcomes)

`record` builds this week's target book for each strategy using the exact same `_target_names` buffer-band logic as the backtest, seeded from the strategy's last recorded book, and appends rows to `ledger/positions.csv`:

```
score_date, strategy, ticker, weight, exposure
2026-07-02, ml, GE, 0.02, 1.0
```

It's **idempotent** — if this week's date is already in the file, it does nothing and returns `False`. That's what lets the weekly cron job re-run safely after a crash without double-recording.

7.2 Settling (after outcomes)

`settle_pending` is the "grade the homework" step. A book recorded on date t is entered at the first close after t and **exits at the first close after the next week's book was recorded** — so each book is held exactly one rebalance period, matching the backtest. A book can only settle once that exit price exists in the data:

```
for i, sd in enumerate(dates[:-1]):        # the last book has no successor yet
    entry = _entry_date(closes.index, sd)
    exit_ = _entry_date(closes.index, dates[i + 1])
    if entry is None or exit_ is None:
        continue                            # exit price doesn't exist yet
    ...
    gross = _book_return(book, closes, entry, exit_)
    cost  = _turnover(book, prior) * config.COST_BPS / 1e4
    new_rows.append({..., "net_return": gross - cost})
```

For each settled week it computes the gross return (weighted sum of each holding's entry→exit return), subtracts the same 10bps × turnover cost the backtest uses, and writes `net_return` to `ledger/performance.csv`. It also writes one **SPY benchmark row** per week, so the record always answers "did we beat just buying the index?" And it's idempotent again — already-settled `(date, strategy)` pairs are skipped.

This is why the dashboard says "1 week recorded, 0 settled." The newest book has no successor yet, so it can't be graded. It takes ~2 weeks per book to settle, and ~15 settled weeks before the record says anything statistically. That wait — until ~October 2026 — is the point of the whole "research pause." The backtest already told its story; now the un-fudgeable forward record gets to referee it.

---

8. Wiring it together — `pipeline.py` and the weekly run

`pipeline.py` is the command-line conductor. Each Friday at 23:00 UTC, cron runs `scripts/weekly.sh`, which calls these in order:

1. `update-data` — refresh the price cache (§2.2).

2. `train` — build features + labels, run all three scorers walk-forward, save scores and feature-importance (§3–4).
3. `backtest` — simulate all three strategies + SPY over the matched window, save equity curves and `metrics.json` (§5–6).
4. `robustness` — re-run under stress: doubled costs, per-calendar-year breakdown, and veterans-only (full-history stocks) to probe survivorship bias. Saves `robustness.json`.
5. `mark` (settle) — grade every book whose window is now complete (§7.2).
6. `record` — write this week's new book, before its outcome exists (§7.1).

Then the shell script commits the ledger to git (three places: local repo, bare mirror, and it pushes), commits the append-only news store, appends a summary to the brain vault, and re-renders the public status page. Every step is idempotent, so a partial failure self-heals on the next run. That's what "fully autonomous" means here — nobody has to touch it.

---

9. The referee's clipboard — diagnostics and the M1 report

The forward record referees the system (§7). But a referee needs a clipboard: notes taken during the game, and a scoring sheet agreed on before it. That's what `diagnostics.py` and `m1_report.py` are. Both were added in July 2026, while the record was one week old and nobody knew anything — deliberately.

9.1 The snapshot — because `outputs/` has amnesia

Every Friday, `train` overwrites `outputs/` — scores, feature importances, the lot. That's fine for operating (you only trade the newest scores), but fatal for diagnosis: come October you'd want to ask "was the model's ranking in July actually related to what happened next?" and "did the features it leaned on drift over the summer?" — and the July artifacts would be long gone.

So `quant_signals/diagnostics.py` runs right after `record` each Friday and archives the **pre-outcome** artifacts into `diagnostics/<score_date>/`: every strategy's scores, the z-scored feature cross-section the champion scored on, and the model's feature importances. Git-tracked, like the ledger — point-in-time history that can't be quietly rewritten. The step is non-fatal by design: a snapshot failure warns and moves on, because the ledger must never be blocked by its own note-taker.

9.2 The M1 report — grading with honest error bars

`quant_signals/m1_report.py` is the pre-registered October read of the record. Its statistical core is three ideas, each an antidote to a specific way of fooling yourself with ~15 numbers:

Bootstrap confidence interval. With ~15 settled weeks, the average "champion minus SPY" difference will not be zero — noise guarantees that. The question is

whether it's distinguishable from zero. The bootstrap answers it without assuming returns are bell-shaped: resample the 15 weekly differences with replacement 10,000 times, compute each resample's mean, and take the middle 90% of those means. If that interval contains zero, the record is consistent with "no edge at all."

Minimum detectable edge. The nastier trap is asking data for an answer it cannot contain. `1.645 × std / √N` says: given this many weeks and this much noise, what's the smallest true weekly edge the test could even detect? At N≈15 that number comes out several times larger than any edge a system like this could plausibly have. The report prints it so the reader sees — in numbers — that M1 is a process check, not a verdict on the edge. (This is also why V-ALPHA below is pre-committed to say "no decision may cite performance.")

Factor-vs-alpha attribution. Suppose the champion did beat SPY. Was that skill, or did it just lean on a well-known factor you could buy for 15 basis points? The report compares the champion against a pre-named battery — SPY (the market), RSP (equal-weight, since an equal-weighted book of 50 names structurally resembles it), and MTUM (a momentum ETF, since momentum features dominate the model). Beating SPY but trailing MTUM earns the label FACTOR-CARRIED, not genius. The battery was named before any outcome existed, so nobody can shop for a flattering benchmark afterwards.

It also tracks **drift** from the snapshots: do feature importances keep the same ranking week to week (a Spearman correlation against both the previous and the first snapshot), and what was the realized rank IC — the correlation between the scores the model published and the returns that then actually happened? A model whose importances lurch around, or whose realized IC sits at zero while the book happens to win, is winning for the wrong reasons.

9.3 The date gate and pre-committed verdicts — handcuffs, on purpose

Two design choices look paranoid and are the whole point:

1. **The gate.** `m1_report` refuses to run on the real ledger before 2026-10-16 (`config.M1_DATE`). During development it only ever saw synthetic fixture data (`tests/m1_fixtures.py` fabricates a fake ledger with a known planted edge, so the math could be tested against known answers). Why? Because if you peek at the record in August, you will react to it — tweak something, stop something, convince yourself of something. At N=5 anything you see is noise. The gate makes peeking a `SystemExit` instead of a temptation.
2. **Pre-committed verdicts.** The report's conclusions were written in July, before any settled data existed, and the code just fills in blanks. V-PROCESS checks the machinery (no missing weeks). V-ALPHA is fixed text: at this N, no promotion, demotion, or sizing decision may cite M1 performance. V-FACTOR is a descriptive label that triggers nothing. V-RESEARCH says research reopens only on pre-named candidates — never "because the record looks good" or

"looks bad," both of which are noise at this sample size. This is the same trick as the label's `shift(-1)`, applied to psychology: decide the rules while you're still ignorant, so the future can't seduce you into moving the goalposts.

The deeper pattern: §3–§5 built walls against data leaking backwards in time; §9 builds the same walls against judgment leaking backwards. Same discipline, different substrate.

---

10. Why this is built the way it is — the deeper lesson

You now understand the machinery. The philosophy is worth stating plainly, because it's the actual product:

- **Everything is relative/market-neutral** — z-scored features, demeaned labels, ranking selection. The system has one small opinion (the regime filter) about the market as a whole and otherwise refuses to guess.
- **Every temporal boundary is guarded** — the `shift(-1)` in the label, the `purge_days` gap in training, the execute-next-day rule in the backtest. These all enforce the same wall: you may only use information you actually had at the time. Break any one and you get a gorgeous fantasy.
- **There is always a control group** — the fixed-weight baseline, the SPY benchmark row, the robustness stress tests. Nothing is trusted on its own.
- **The forward record is the only truth.** Three research rounds (tuning, a bigger universe, extra features) each looked better in the backtest and then failed a strict held-out test. That's not failure of the method — that's the method working, catching self-deception before it cost real money. The single hardest and most valuable discipline in the entire project is doing nothing while the paper record accumulates.

If you take one idea from this document: **in trading, the easy part is making a backtest look good, and the entire craft is building the machinery that stops you from believing it.** Everything above is that machinery.

Generated for Aditya. Pairs with `usage.md` (how to operate it) and `now.md` (current state). To regenerate the PDF: `~/.doctools/bin/python ~/.doctools/md2pdf.py docs/internals.md out.pdf "Title"`.